
CO4AT1

CSA0918- JAVA PROGRAMMING

NAME: JEYA HARSHINI R

REG NO: 192424051

Java Industry Problem-Solving: Eliminating Flicker Using Double Buffering

1. Title

Flicker-Free Live Floor Map Using Double Buffering in Java AWT

2. Problem Statement

A retail company's point-of-sale training application displays a **live seating/floor map** using the Graphics object obtained inside the paint() method. Developers observe noticeable **flickering whenever the application window is resized**.

The problem must be diagnosed based on Java AWT's painting mechanism, and a **double-buffering technique** must be implemented to produce smooth, flicker-free rendering.

3. Problem Diagnosis

In Java AWT, the paint(Graphics g) method is **not called only when the programmer explicitly requests it**.

The AWT system may invoke paint() whenever a component needs to be redrawn, such as when:

- The window is resized.
- The window is uncovered after being hidden.
- Another window moves away from it.
- repaint() is requested.
- The operating system determines that the component needs refreshing.

What happens during resizing?

When the window is resized, AWT may repeatedly request the component to repaint.

The process can effectively look like:

Window Resize

↓

AWT requests repaint

↓

paint(Graphics g)

↓

Background is cleared

↓

Floor map is drawn

↓

AWT displays result

↓

Another repaint request

↓

Background cleared again

↓

Floor map drawn again

The user can briefly see the **cleared/partially drawn state** before the complete map is displayed.

This rapid alternation between the background and the newly drawn map produces the visible **flicker**.

4. Role of paint() and repaint()

paint(Graphics g)

paint() is responsible for rendering the component.

For example:

```
public void paint(Graphics g) {  
    g.drawRect(50, 50, 200, 100);  
}
```

The Graphics object represents the drawing surface provided by AWT.

However, paint() should **not be assumed to run only once**. The system can call it repeatedly whenever the component needs to be refreshed.

repaint()

repaint() does not normally perform the drawing immediately.

Instead, it tells AWT:

"This component needs to be painted again."

For example:

```
repaint();
```

AWT schedules a painting operation, which eventually results in paint() being called.

Therefore:

```
repaint()
```

↓

AWT schedules painting

↓

```
paint(Graphics g)
```

↓

Drawing occurs

Calling repaint() repeatedly for a live floor map can therefore cause paint() to execute frequently.

5. Root Cause of Flickering

The primary problem is that the application is drawing the floor map **directly onto the screen during every repaint**.

During each repaint:

1. The visible drawing surface may be cleared.
2. The floor map is redrawn.
3. The user can see intermediate drawing operations.
4. During rapid resizing, this happens repeatedly.

Hence, the user sees a visible flashing effect.

Root cause

Direct rendering to the visible screen during repeated repaint operations exposes intermediate drawing states to the user.

6. Industry Solution — Double Buffering

The standard solution is **double buffering**.

Instead of drawing directly onto the visible screen, the application maintains an **off-screen image (buffer)**.

The complete floor map is first rendered into this invisible buffer.

Only after the entire drawing is complete is the buffer copied to the screen in one operation.

Without double buffering

Graphics

↓

Screen

↓

Draw table

↓

Draw chair

↓

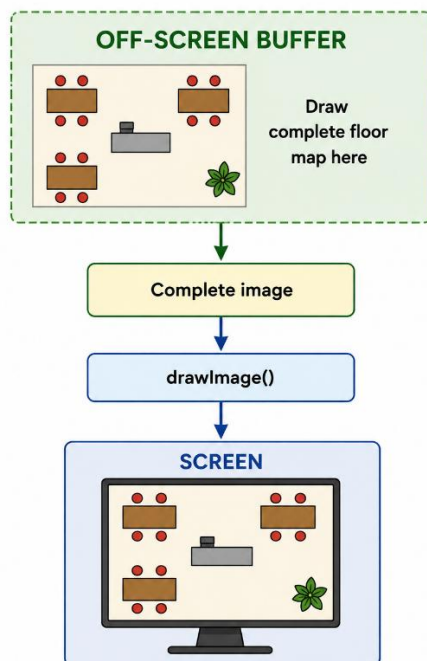
Draw counter

↓

Draw labels

The user can see these intermediate stages.

With double buffering



The user sees the completed image rather than the individual drawing operations.

7. Algorithm

1. Create an off-screen Image buffer.
 2. Obtain a Graphics object associated with the buffer.
 3. Draw the complete floor map onto the off-screen buffer.
 4. Clear the buffer before drawing the next frame.
 5. Draw all seating and floor-map elements into the buffer.
 6. Copy the completed buffer to the screen using drawImage().
 7. Call repaint() when the map needs to be refreshed.
 8. Repeat the process for subsequent frames.
-

8. Java Implementation

Note: Java Applets are obsolete/deprecated in modern Java environments. The following implementation demonstrates the **AWT applet painting and double-buffering concept** requested by the problem.

```
File    Edit    View

import javax.swing.*;
import java.awt.*;

public class FloorMap extends JPanel {

    private Image buffer;
    private Graphics bufferGraphics;

    public FloorMap() {
        setPreferredSize(new Dimension(400, 300));
        setBackground(Color.WHITE);
    }

    @Override
    protected void paintComponent(Graphics g) {

        super.paintComponent(g);

        // Create off-screen buffer
        if (buffer == null ||
            buffer.getWidth(this) != getWidth() ||
            buffer.getHeight(this) != getHeight()) {

            buffer = createImage(getWidth(), getHeight());
            bufferGraphics = buffer.getGraphics();
        }

        // Clear buffer
        bufferGraphics.setColor(Color.WHITE);
        bufferGraphics.fillRect(
            0, 0, getWidth(), getHeight()
        );

        // Draw title
        bufferGraphics.setColor(Color.BLACK);
        bufferGraphics.drawString(
            "Retail Store Floor Map", 50, 40
        );

        // Draw tables
        bufferGraphics.setColor(Color.BLUE);

        bufferGraphics.fillRect(
            50, 80, 120, 60
        );

        bufferGraphics.fillRect(
            220, 80, 120, 60
        );

        bufferGraphics.fillRect(
            50, 180, 120, 60
        );

        bufferGraphics.fillRect(
            50, 80, 120, 60
        );

        bufferGraphics.fillRect(
            220, 80, 120, 60
        );

        bufferGraphics.fillRect(
            50, 180, 120, 60
        );

        // Draw chairs
        bufferGraphics.setColor(Color.RED);

        bufferGraphics.fillOval(
            80, 150, 30, 30
        );

        bufferGraphics.fillOval(
            260, 150, 30, 30
        );

        // Copy complete buffer to screen
        g.drawImage(
            buffer,
            0,
            0,
            this
        );
    }

    public static void main(String[] args) {

        JFrame frame = new JFrame(
            "Retail Store Floor Map"
        );

        FloorMap map = new FloorMap();

        frame.add(map);
        frame.pack();

        frame.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE
        );

        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

9. Important Part of the Code

The key part is:

```
buffer = createImage(getWidth(), getHeight());
```

```
bufferGraphics = buffer.getGraphics();
```

This creates an **off-screen drawing surface**.

Instead of:

```
g.drawRect(...);
```

```
g.drawString(...);
```

the application performs the drawing using:

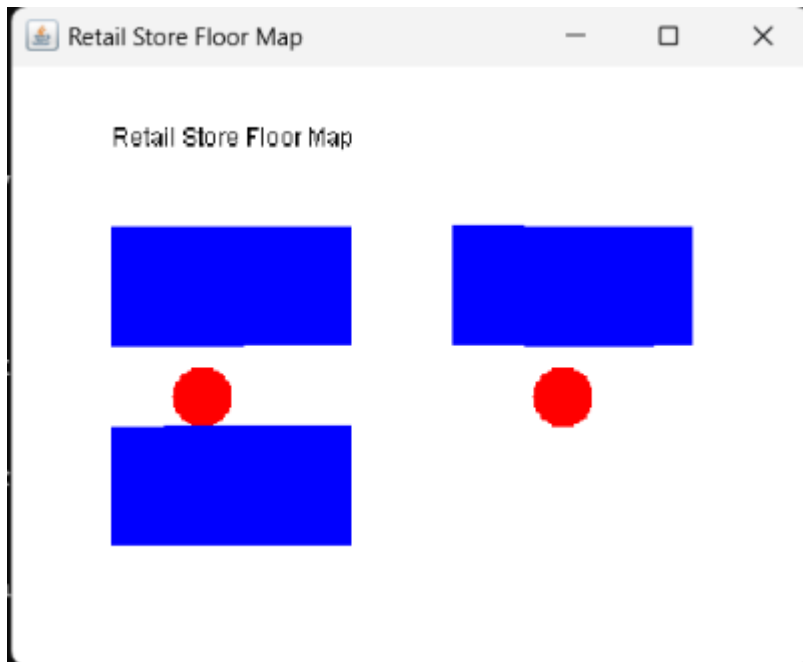
```
bufferGraphics.drawRect(...);
```

```
bufferGraphics.drawString(...);
```

After the complete map has been created:

```
g.drawImage(buffer, 0, 0, this);
```

copies the completed image onto the visible screen.



10. Why update() Is Overridden

AWT's default update(Graphics g) behavior can clear the background before calling paint().

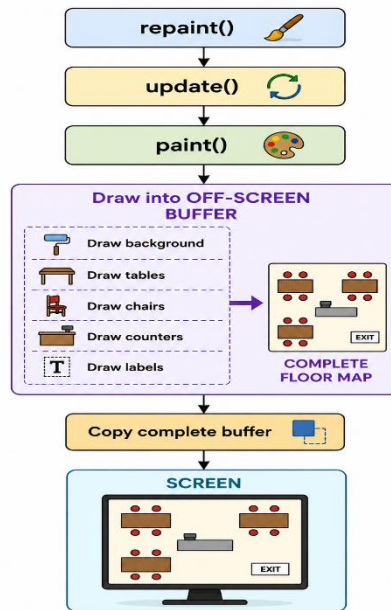
That background-clearing operation can itself contribute to flickering.

Therefore, the program overrides:

```
public void update(Graphics g) {  
    paint(g);  
}
```

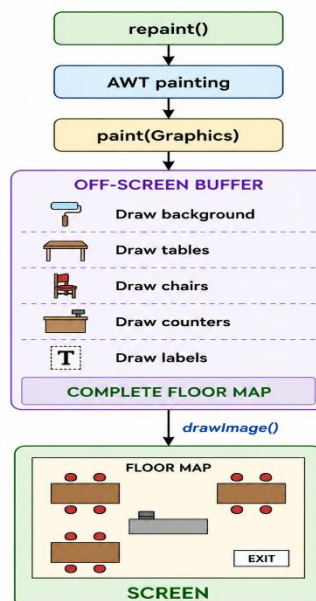
This prevents the default update process from unnecessarily clearing the visible screen before the buffered image is drawn.

The rendering becomes:



11. Working of Double Buffering

The complete process is:



The important idea is:

Complete first, display second.

12. Comparison

Direct Rendering

Draws directly to screen

Intermediate drawing can be visible

Can produce flicker

Poor for rapidly changing graphics

Resizing can expose drawing stages

Simple implementation

Double Buffering

Draws to off-screen buffer

Intermediate drawing remains hidden

Greatly reduces flicker

Suitable for animations/live maps

Completed frame is copied to screen

Requires an additional buffer

13. Industry-Level Approach

For a real modern Java desktop application, the same principle is commonly handled by the GUI framework rather than manually implementing an old-style Applet.

For example, in **Swing**, components such as JPanel support optimized painting and Swing provides double-buffering facilities.

For a custom component, the drawing would typically be placed in:

```
@Override
```

```
protected void paintComponent(Graphics g) {
```

```
    super.paintComponent(g);
```

```
    // Draw floor map
```

```
}
```

A modern Swing application can therefore use the framework's painting infrastructure rather than relying on the deprecated Applet model.

14. Expected Result

After implementing double buffering:

- The floor map is rendered in an off-screen buffer.
- Resizing causes repaint operations as before.

- The user does not see the intermediate drawing process.
- The completed map is copied to the screen in a single operation.
- Visible flickering is eliminated or greatly reduced.
- The floor map appears smooth during repeated updates.

15. Conclusion

The flickering occurs because **AWT may invoke paint() repeatedly during resizing and repaint requests**, and direct drawing exposes intermediate rendering states to the user. The default update/clear behavior can make this more noticeable.

The industry-standard solution is **double buffering**, where the entire frame is first rendered into an off-screen image and then copied to the visible component using `drawImage()`. This ensures that the user sees a **complete frame instead of partially rendered frames**, resulting in smooth and flicker-free graphics.

Final Answer in One Line

Use an off-screen Image as a double buffer, render the complete floor map into it inside paint(), and then copy the completed buffer to the screen with drawImage(); this prevents the intermediate clearing and drawing states caused by repeated paint()/repaint() operations from becoming visible.